University course pack

Teaching databases and data engineering at university

A 14-week, 3-credit module in database systems and data engineering for third-year computer science or information systems students. Prerequisites: programming and discrete mathematics. The four Darsi lectures anchor weeks 3, 6, 8 and 11.

Learning outcomes

1. Design normalised relational schemas from functional dependencies.
2. Write correct SQL and read query plans.
3. Choose indexes from selectivity and workload.
4. Explain isolation anomalies and distributed trade-offs.
5. Build data pipelines with automated quality checks and documentation.

14-week syllabus

Week	Topic	In class and out
1	Data models and the relational model	Reading: Codd 1970
2	Relational algebra and SQL	Problem set 1
3	Functional dependencies and normalisation	Lab 1: break it, then fix it
4	Entity–relationship design	Design assignment
5	Storage and files	Problem set 2
6	Indexing and B-trees	Lab 2: query tuning
7	Query processing and optimisation	Midterm exam
8	Transactions and isolation	Lab 3: concurrency anomalies
9	Recovery and logging	Problem set 3
10	Replication, partitions and CAP	Case-study debate
11	Data quality and pipelines	Project start: real dataset
12	Data warehousing and denormalisation	Project checkpoint
13	Privacy and responsible data	Datasheet draft
14	Project presentations and review	Final exam

University course pack

Lecture slide outline

Lecture 1: Normalisation

- A database that disagrees with itself
- Relations and keys
- Functional dependencies and closure
- 2NF, 3NF and BCNF
- Activity: break it, then fix it

Lecture 2: Indexing

- Eight seconds to three milliseconds
- Pages and scans
- B+-trees and fan-out
- Selectivity and write cost
- Activity: tune one query

Lecture 3: Transactions and CAP

- One seat sold twice
- ACID
- Isolation levels and anomalies
- Partitions and CAP
- Activity: interleave and fix

Lecture 4: Data quality

- A 112% attendance report
- Dimensions of quality
- Tests and constraints
- Lineage and datasheets
- Activity: clean a real dataset

University course pack

Problem set

1. R(A, B, C, D) with AB → C, C → D and D → A. Find all candidate keys.

2. Decompose Enrol(student, course, grade, lecturer, phone) with course → lecturer and lecturer → phone into 3NF and show the join is lossless.

3. A table has 10^7 rows in pages of 100 rows. A B+-tree has fan-out 200. Compare pages read by a full scan and an index lookup returning one row.

4. Two doctors are on call; a rule says at least one must be. Under snapshot isolation both go off call at once. Name the anomaly and give a fix.

5. For course registration and for the library search catalogue, say which you would favour during a partition, consistency or availability, and why.

6. Write three SQL quality checks for an enrolment table and one constraint that makes each unnecessary.

Solutions

1. AB, CB and DB. B appears on no right-hand side, so every key contains B; $\{A,B\}^+$, $\{C,B\}^+$ and $\{D,B\}^+$ are all ABCD, and B alone is not a key.

2. Enrol(student, course, grade), Course(course, lecturer), Lecturer(lecturer, phone). Each join is on an attribute that is a key of one side, so it is lossless.

3. Scan: 10^5 pages. Index: $\lceil \log_{200} 10^7 \rceil = 4$ levels plus one data page, about 5 pages.

4. Write skew. Use serializable isolation, or lock the rows read with SELECT … FOR UPDATE before updating.

5. Registration: consistency, since overbooking seats is unacceptable. Catalogue: availability, since slightly stale search results are harmless.

6. Duplicates via GROUP BY … HAVING COUNT(*) > 1 (PRIMARY KEY); grades outside 0–100 (CHECK); courses not in the course table via LEFT JOIN … IS NULL (FOREIGN KEY).

Lab handout: Lab 3: Concurrency anomalies

Goal

Reproduce, name and fix isolation anomalies on a real database using two sessions.

Steps

1. Open two sessions on the course-registration schema.
2. Reproduce a lost update on the last seat under read committed.
3. Reproduce write skew under snapshot isolation.
4. Fix each with the smallest change and verify.
5. Write a note on what registration needs during a partition.

Hand in

Session transcripts, fixes and a one-page note.

University course pack

Assessment rubric

Criterion	Excellent	Good	Developing	Not yet
Schema design	BCNF or justified trade-off	3NF with keys	Mostly normalised	Redundant
Query and index reasoning	Justified and measured	Indexes justified	Plans read	No plans read
Concurrency	Fixed with trade-offs argued	Named and fixed	Reproduced	Anomaly not seen
Data quality	Plus log and datasheet	Checks and constraints	A few checks	No checks

Reading list

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387.
The relational model.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.
Core text.

Bayer, R., & McCreight, E. (1972). Organization and maintenance of large ordered indexes. Acta Informatica, 1(3), 173–189.
The B-tree.

Selinger, P. G., et al. (1979). Access path selection in a relational database management system. Proceedings of SIGMOD '79, 23–34.
Cost-based optimisation.

Berenson, H., et al. (1995). A critique of ANSI SQL isolation levels. Proceedings of SIGMOD '95, 1–10.
Isolation anomalies.

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News, 33(2), 51–59.
The CAP theorem.

Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly.
Theory to practice.

Gebru, T., et al. (2021). Datasheets for datasets. Communications of the ACM, 64(12), 86–92.
Documenting data.